PATTERNS DE RÉSILIENCE · M2 LDF · PROJET FINAL

# ShopFlow — Architecture résiliente complète

Dossier d'architecture : contexte, risques, patterns, observabilité, chaos engineering, déploiement

**Étudiante :** Yasmine Ghouar · **Formatrice :** Suzanne KOKOS · **Date :** 2 novembre 2026

Ce dossier prolonge les TP 7 à 10 (rate limiting, health checks, chaos engineering, déploiement) : le service `shopflow_api` devient le cœur d'une architecture de cinq services HTTP, instrumentée avec Prometheus et Grafana, et éprouvée par sept expériences de chaos automatisées, chacune jouée **sans** puis **avec** patterns. Tous les chiffres de ce document proviennent de mesures réelles (fichiers JSON dans `chaos/results/`).

**61,2 % → 95,5 %**
SLI moyen pendant les 7 pannes, sans → avec patterns

**10**
patterns implémentés, chacun relié à un risque de l'AMDEC

**37 → 0**
paiements débités sans confirmation (E5), sans → avec idempotence

**3 + 1**
faiblesses découvertes par le chaos et corrigées (re-mesurées) + 1 hypothèse invalidée

## Sommaire

1. Contexte, objectifs et SLO
2. Architecture
3. Analyse des risques : dépendances, disponibilité théorique, SPOF, AMDEC
4. Patterns de résilience et paramètres justifiés
5. Observabilité : métriques, tableau de bord, alertes
6. Déploiement : Docker Compose et Kubernetes
7. Chaos engineering : protocole et résultats
8. Fiches d'expérience
9. Faiblesses découvertes et corrections (avant / après)
10. Limites et perspectives

Dépôt : `projet_final/` — tout lancer : `docker compose up -d --build` · tests : `python -m unittest discover tests` · expériences : `python chaos/chaos.py` puis `python chaos/rapport.py` · ce dossier : `python docs/build_dossier.py` · détails : `README.md`.

## 1. Contexte, objectifs et SLO

ShopFlow est une boutique en ligne simplifiée. Deux parcours comptent pour l'utilisateur : **consulter une fiche produit** (stock disponible et recommandations) et **payer sa commande**. Ces parcours reposent sur trois services tiers que l'on ne maîtrise pas : le stock, le moteur de recommandations et le prestataire de paiement. Ils sont simulés par le `flaky_service` du kit du jour 2, dont on peut piloter les pannes (erreurs, latence, indisponibilité, réponses perdues).

L'objectif du projet est de démontrer, preuves à l'appui, que l'application **continue de rendre un service acceptable** face à des pannes réalistes, et qu'elle est **prête à être exploitée** : déploiement reproductible, sondes de santé, métriques, tableau de bord et alertes.

### Indicateur et objectif de niveau de service

**SLI** : part des requêtes métier (`GET /products/...`, `POST /checkout`) servies avec un code **2xx en moins de 1 s**, réponses dégradées comprises (définition du TP 9). Les 429 renvoyés volontairement à un client qui dépasse son quota sont exclus : c'est le comportement attendu.
**SLO** : 99 % sur 30 jours glissants, soit un **budget d'erreur de 1 %** (≈ 7 h 18 min par mois).
**Suivi séparé de la qualité** : part de réponses dégradées (stock issu du cache, page sans recommandations). Le TP 9 a montré qu'un SLI « servi » peut rester haut alors que la dépendance est entièrement en panne : on suit donc aussi la part dégradée.

### Exigences du projet et réponse apportée

| Exigence | Réalisation |
|---|---|
| ≥ 3 services HTTP dont une dépendance instable | 5 services : gateway NGINX, `shopflow-api` (2 réplicas), stock, reco, payment (3 dépendances instables) |
| Analyse des risques | Graphe de dépendances, disponibilité théorique, SPOF, AMDEC à 12 modes de défaillance (section 3) |
| ≥ 6 patterns reliés à l'AMDEC | 10 patterns : timeout, retry, circuit breaker, bulkhead, fallback, cache résilient, dégradation gracieuse, rate limiting, load shedding, health checks |
| Observabilité | ≥ 1 métrique Prometheus par pattern, tableau de bord Grafana provisionné, alertes SLO multi-fenêtres |
| ≥ 5 expériences de chaos | 7 expériences automatisées, hypothèse, critère d'arrêt, résultats avant / après |
| Déploiement | Docker Compose (une commande) ; manifestes Kubernetes avec 3 sondes et PodDisruptionBudget (bonus) |

## 2. Architecture

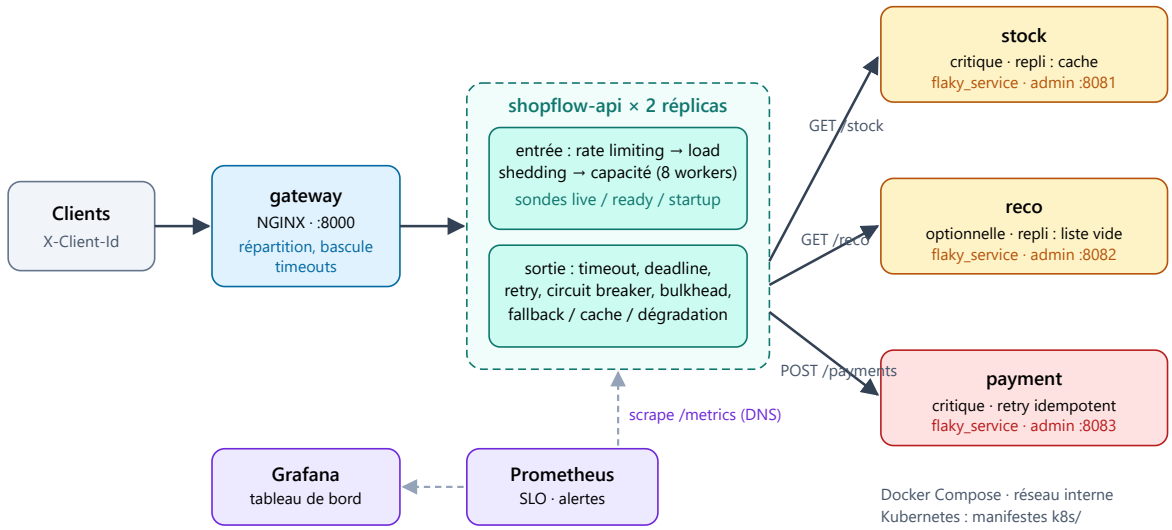



Figure 1 — Architecture de ShopFlow : flux HTTP (traits pleins) et collecte des métriques (pointillés)

### Rôle de chaque service

| Service | Technologie | Responsabilité | Criticité |
|---|---|---|---|
| gateway | NGINX 1.27 | Point d'entrée unique ; répartit sur les réplicas (DNS Docker réévalué toutes les 5 s) ; bascule sur l'autre réplica si une connexion échoue | Critique |
| shopflow-api ×2 | Python 3.12, stdlib | `GET /products/<sku>` (stock + reco), `POST /checkout` (paiement), sondes, `/metrics`. Porte tous les patterns | Critique |
| stock | flaky_service | Quantité disponible par SKU. Dépendance critique mais **tolérante à la fraîcheur** : une valeur de quelques secondes reste utile | Critique, avec repli |
| reco | flaky_service | Recommandations associées au produit. **Optionnelles** : la page reste utile sans elles | Optionnelle |
| payment | flaky_service | Débit de la commande. Aucune valeur de repli possible ; opération **non idempotente** par nature | Critique, sans repli |
| Prometheus / Grafana | v2.54 / v11.2 | Collecte (découverte DNS des réplicas), règles SLO, alertes ; tableau de bord | Hors chemin des requêtes |

### Chaîne de traitement d'une requête dans l'API

À l'entrée, chaque requête traverse : **rate limiting** (token bucket par `X-Client-Id`) → **load shedding** (file d'attente bornée, places réservées au checkout) → **capacité de l'instance** (8 workers, qui simulent un pool de connexions). À la sortie, chaque appel à une dépendance est enveloppé, dans l'ordre vu au jour 2 : `fallback( retry( circuit_breaker( bulkhead( appel_http(timeout) ) ) ) )`, avec en plus un **budget de temps global** (deadline) partagé par toutes les tentatives. La variable `PATTERNS=off` désactive tous ces mécanismes : timeout naïf de 10 s, aucun retry, aucun repli. C'est la configuration « avant » des expériences.

## 3. Analyse des risques

### 3.1 Graphe de dépendances

**Page produit** : client → gateway → API → **stock** (requis) + **reco** (optionnelle).
**Checkout** : client → gateway → API → **payment** (requis, non idempotent).

Sans patterns, toutes les flèches sont des dépendances **dures en série** : la page produit tombe si la reco tombe, alors que les recommandations n'ont aucune valeur critique. Avec patterns, la reco devient une dépendance **souple** (dégradation) et le stock une dépendance **dure mais avec repli** (cache de la dernière valeur connue).

| Composant | Dispo. supposée | Justification |
|---|---|---|
| gateway | 99,9 % | processus simple, sans état |
| réplica API | 99,5 % | crashs, déploiements |
| stock | 99,0 % | dépendance instable |
| reco | 98,0 % | service non critique, moins soigné |
| payment | 99,0 % | prestataire externe |
| couverture du cache | 95 % | SKU déjà consultés < 5 min |

### 3.2 Disponibilité théorique

| Parcours | Sans patterns (série, 1 réplica) | Avec patterns | Indisponibilité / mois |
|---|---|---|---|
| Page produit | $A_{gw}{\cdot}A_{api}{\cdot}A_{stock}{\cdot}A_{reco}$ = 0,999 × 0,995 × 0,99 × 0,98 = **96,44 %** | $A_{gw} \cdot [1-(1-A_{api})^2] \cdot [1-(1-A_{stock})(1-0{,}95)]$ = 0,999 × 0,999975 × 0,9995 = **99,85 %** | 26,0 h → 1,1 h |
| Checkout | 0,999 × 0,995 × 0,99 = **98,41 %** | 0,999 × 0,999975 × 0,99 = **98,90 %** | 11,6 h → 8,0 h |

Le calcul montre où les patterns agissent : la redondance de l'API et le cache font passer la page produit de 96,4 % à 99,85 %, et la dégradation retire la reco de la chaîne. En revanche, le **checkout reste borné par la disponibilité du paiement** (99 %) : le retry corrige les erreurs *transitoires*, pas une panne franche du prestataire, et aucun repli n'est possible pour un débit. Le checkout ne peut donc pas tenir 99 % si le prestataire est lui-même à 99 %. C'est une limite assumée (section 10).

### 3.3 Points de défaillance uniques (SPOF)

- **gateway NGINX** (instance unique) : sa perte coupe tout. En Kubernetes, il est remplacé par un Service et un Ingress répliqués.
- **payment** : critique, sans repli. Atténuation : circuit breaker et réponse 503 + `Retry-After` rapides (on n'attend pas 10 s), alerte.
- **hôte Docker unique** en Compose : les 2 réplicas partagent la même machine (question 3 du TP 10). Kubernetes répartit les pods sur des nœuds et des zones.
- **stock** : SPOF de données, atténué par le cache stale-if-error (≤ 5 min) ; un SKU jamais consulté reste sans valeur (« stock inconnu »).
- Prometheus et Grafana ne sont **pas** sur le chemin des requêtes : leur perte aveugle l'exploitation sans couper le service.

### 3.4 AMDEC

Gravité (G), occurrence (O) et non-détection (D) sont notées de 1 à 10 ; criticité C = G × O × D. La colonne C' donne la criticité estimée **après** mise en place des patterns, que les expériences de la section 7 viennent vérifier.

| # | Mode de défaillance | Effet sans protection | G | O | D | C | Pattern(s) | C' | Exp. |
|---|---|---|---|---|---|---|---|---|---|
| R1 | Erreurs transitoires du stock (503) | page produit en erreur | 6 | 7 | 3 | 126 | retry + backoff exponentiel + jitter | 18 | E1 |
| R2 | Stock indisponible | toutes les pages en erreur | 8 | 4 | 2 | 64 | circuit breaker + cache résilient | 16 | E2 |
| R3 | Stock lent (latence) | threads bloqués, saturation, panne en cascade | 8 | 5 | 6 | 240 | timeout + deadline + bulkhead + breaker | 32 | E3 |
| R4 | Reco lente ou en panne | page produit lente ou en erreur | 5 | 6 | 5 | 150 | dégradation gracieuse + timeout court + bulkhead | 15 | E4 |
| R5 | Paiement : réponse perdue après débit | double débit au réessai, ou commande « perdue » | 9 | 3 | 8 | 216 | retry + clé d'idempotence de bout en bout | 27 | E5 |
| R6 | Paiement indisponible | clients bloqués 10 s, threads épuisés | 9 | 3 | 3 | 81 | circuit breaker + 503 Retry-After + alerte | 54 | E5 |
| R7 | Client abusif / robot | capacité monopolisée, latence pour tous | 6 | 5 | 4 | 120 | rate limiting par client (429) | 24 | E6 |
| R8 | Pic de trafic légitime | files d'attente, latence > SLO pour tous | 7 | 4 | 5 | 140 | load shedding prioritaire (503) | 42 | E6 |
| R9 | Crash d'une instance API | requêtes en erreur jusqu'au redémarrage | 7 | 3 | 3 | 63 | redondance + health checks + restart | 21 | E7 |
| R10 | Arrêt / déploiement coupant des requêtes | erreurs à chaque mise en production | 5 | 6 | 5 | 150 | readiness + drain 5 s, rolling update, PDB | 20 | TP 8/10 |
| R11 | Gateway qui éjecte des instances saines | erreurs amplifiées, panne totale | 9 | 3 | 7 | 189 | bascule sur erreurs de connexion seulement + alerte | 27 | E1* |
| R12 | Tempête de retries | charge ×3 sur une dépendance déjà fragile | 7 | 4 | 6 | 168 | retries bornés, jitter, deadline, breaker | 28 | E1, E3 |

* R11 n'était pas dans l'AMDEC initiale : il a été **découvert** par l'expérience E1 (section 9).

## 4. Patterns de résilience et paramètres justifiés

| Pattern | Où | Paramètres | Justification | Risque |
|---|---|---|---|---|
| Timeout | client.py, chaque appel | stock 0,3 s · reco 0,25 s · payment 1 s | p99 nominal du stock ≈ 0,08 s : 0,3 s laisse 4× de marge. Reco plus courte car optionnelle. Payment plus long car rare et coûteux à abandonner | R3, R4 |
| Deadline (budget global) | resilience.Dependance | stock 0,8 s · payment 2,5 s | Les tentatives successives ne doivent jamais dépasser le SLO de 1 s (correctif issu de E3, section 9) | R3, R12 |
| Retry | resilience.Dependance | 3 tentatives, backoff $0{,}05 \cdot 2^i$ avec full jitter, respecte Retry-After | 30 % d'erreurs indépendantes → $0{,}3^3$ = 2,7 % d'échec résiduel ; le jitter évite la synchronisation des clients (TP 3, TP 7) | R1, R5 |
| Circuit breaker | resilience.CircuitBreaker | fenêtre 20 appels, ≥ 50 % d'échecs, volume min. 10, ouvert 5 s, 1 essai en semi-ouvert | Taux d'échec plutôt que compteur consécutif : insensible au bruit (30 % d'erreurs ne l'ouvrent pas, une panne franche l'ouvre en moins d'une seconde) | R2, R3, R6 |
| Bulkhead | resilience.Bulkhead | stock 10 · reco 4 · payment 6 places, attente 50 ms | Une dépendance lente ne peut pas occuper plus que sa part des 8 workers : la reco (4) ne peut jamais bloquer le stock | R3, R4 |
| Fallback | page_produit, checkout | stock → cache ; reco → [] ; payment → 503 + Retry-After | Repli adapté à la valeur métier de chaque donnée | R2, R4, R6 |
| Cache résilient | resilience.CacheResilient | stale-if-error, âge max. 300 s | Une quantité vieille de quelques minutes est préférable à une erreur ; la donnée nominale reste toujours fraîche | R2 |
| Dégradation gracieuse | page_produit | en-tête `X-Degraded`, champ `degrade` | La page reste utile sans reco ; le client sait que la réponse est dégradée | R4 |
| Rate limiting | rate_limit.TokenBucket (TP 7) | par client : 20 jetons, 10 jetons/s ; 429 + Retry-After | Un utilisateur réel fait ≈ 1 à 5 req/s : 10/s avec une rafale de 20 ne gêne que les robots | R7 |
| Load shedding | rate_limit.LoadShedder (TP 7) | 40 requêtes en cours par réplica, 25 % réservées au checkout ; 503 + Retry-After: 1 | Loi de Little : 8 workers × (0,6 s de budget d'attente / 0,12 s de service) ≈ 40 (recalibré après E6, section 9) | R8 |
| Health checks | health.py (TP 8) | startup (3 s), readiness locale, liveness processus, drain 5 s sur SIGTERM | La readiness **n'inclut plus** les dépendances partagées : sinon toutes les instances quitteraient le pool ensemble (TP 8, question 2) | R9, R10 |

### Idempotence du paiement

Le client fournit une clé `Idempotency-Key` par panier (générée par l'API s'il n'en fournit pas). L'API la transmet au prestataire à chaque tentative, qu'elle vienne du retry interne ou d'un réessai de l'utilisateur. Le prestataire renvoie le paiement déjà enregistré au lieu de débiter une seconde fois. Sans cette clé, réessayer un POST dont la réponse a été perdue revient à **débiter deux fois** (expérience E5).

## 5. Observabilité

### 5.1 Métriques exposées (`GET /metrics`, format Prometheus)

Le module `metrics.py` (bibliothèque standard uniquement) expose des compteurs, des jauges et des histogrammes. Prometheus découvre chaque réplica par le DNS Docker ( `dns_sd_configs` sur le nom de service `api` ), toutes les 5 s.

| Pattern | Métrique(s) |
|---|---|
| SLI / latence | `shopflow_http_requests_total{route,code}`, `shopflow_http_request_duration_seconds{route,classe}` (histogramme) |
| Timeout / retry | `shopflow_timeouts_total{dep}`, `shopflow_retries_total{dep}`, `shopflow_dependency_calls_total{dep,issue}` |
| Circuit breaker | `shopflow_circuit_state{dep}` (0/1/2), `shopflow_circuit_transitions_total{dep,vers}` |
| Bulkhead | `shopflow_bulkhead_in_use{dep}`, `shopflow_bulkhead_rejected_total{dep}` |
| Fallback / cache / dégradation | `shopflow_fallback_total{dep,type}`, `shopflow_cache_events_total`, `shopflow_degraded_responses_total` |
| Rate limiting / load shedding | `shopflow_rate_limited_total{route}`, `shopflow_load_shed_total{route,priorite}`, `shopflow_inflight_requests` |
| Health checks | `shopflow_ready`, `up{job="shopflow-api"}`, `shopflow_patterns_enabled` |

## 5.2 Tableau de bord Grafana

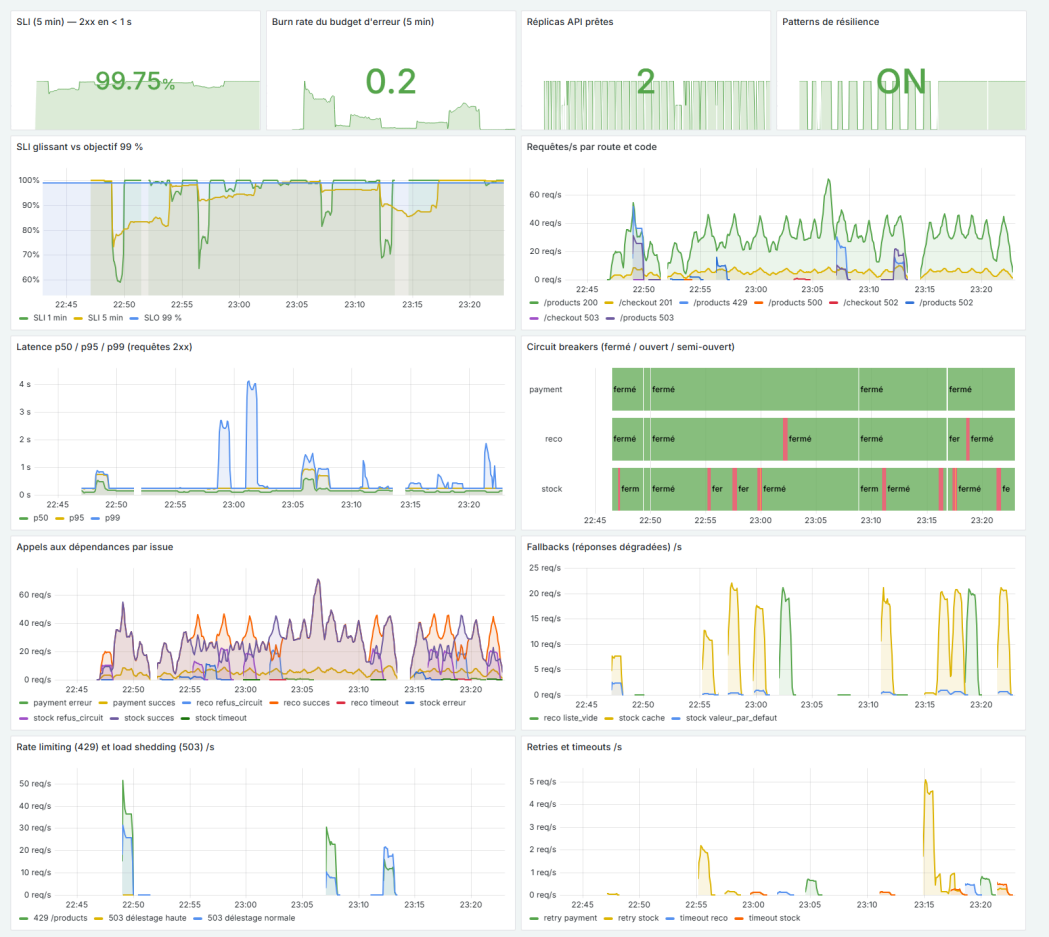



Figure 2 — Tableau de bord « ShopFlow — Résilience & SLO » (provisionné automatiquement), capturé pendant la campagne d'expériences

## 5.3 Règles d'alerte sur le SLO

Des règles d'enregistrement calculent le SLI sur 1 min, 5 min et 1 h, puis le **taux de consommation du budget** (burn rate = taux d'erreur / 1 %). On applique la méthode multi-fenêtres : une fenêtre longue garantit que le problème est significatif, une fenêtre courte que l'alerte retombe vite une fois le problème résolu. Un burn rate de 14,4 consomme 2 % du budget mensuel en une heure (fenêtres réduites à 5 min / 1 min pour rester observables à l'échelle d'un TP).

```
- name: shopflow-slo-alerts
  rules:
    - alert: ShopFlowSLOBurnRateRapide
      # Budget mensuel consommé 14,4x trop vite (2 % du budget en 1 h) : on réveille l'astreinte.
      expr: shopflow:burn_rate:5m > 14.4 and shopflow:burn_rate:1m > 14.4
      for: 1m
      labels:
        severity: page
      annotations:
        summary: "SLO ShopFlow en danger : burn rate {{ $value | printf \"%.1f\" }}x"
        description: "Moins de 99 % des requêtes sont servies en 2xx sous 1 s (fenêtres 5 min et 1 min)."
```

Trois alertes complètent le SLO : `ShopFlowCircuitOuvert` (dépendance en panne, service dégradé), `ShopFlowInstanceIndisponible` (réplica injoignable) et `ShopFlowDelestageEleve` (> 5 % de 503 : la capacité est insuffisante).

### 5.4 Preuve de déclenchement des alertes

Les pannes de 20 s des expériences sont volontairement plus courtes que les délais de confirmation des alertes (30 s pour le breaker, 1 min pour le burn rate) : on ne réveille pas l'astreinte pour un incident déjà résolu. Pour vérifier la chaîne d'alerte, `chaos/demo_alerte.py` arrête le stock pendant 150 s, d'abord sans patterns, puis avec :

```
23:23:38 stock DOWN (patterns OFF) pendant 150 s
23:23:58   t+   20 s  alerte ShopFlowSLOBurnRateLent  -> pending
23:24:08   t+   30 s  alerte ShopFlowSLOBurnRateRapide  -> pending
23:25:08   t+   90 s  alerte ShopFlowSLOBurnRateRapide  -> firing
23:26:38 stock DOWN (patterns ON) pendant 150 s
23:26:43   t+    5 s  alerte ShopFlowSLOBurnRateRapide  -> firing
23:26:43   t+    5 s  alerte ShopFlowSLOBurnRateLent  -> pending
23:26:48   t+   10 s  alerte ShopFlowCircuitOuvert stock -> pending
23:27:13   t+   35 s  alerte ShopFlowCircuitOuvert stock -> firing
23:28:53   t+  135 s  alerte ShopFlowSLOBurnRateLent  -> firing
```

Sans patterns, le SLO s'effondre : l'alerte rapide passe en *pending* à t+30 s et en *firing* à t+90 s (30 s pour que la fenêtre de 1 min dépasse le seuil, puis la durée `for: 1m`). Avec patterns, le SLI est préservé ; c'est `ShopFlowCircuitOuvert` qui signale le mode dégradé (firing à t+35 s). Les alertes de burn rate encore actives au début de la phase 2 sont le résidu de la phase 1, toujours présente dans les fenêtres de 5 min et de 1 h.

## 6. Déploiement

### 6.1 Docker Compose (obligatoire) — une commande

`docker compose up -d --build` construit deux images (API et dépendances : Python *slim*, utilisateur non root `uid 10001`, `CMD` en forme exec, `HEALTHCHECK`) et démarre 8 conteneurs. Quelques choix notables :

- `depends_on: condition: service_healthy` : l'API attend que ses dépendances soient saines, et le gateway attend l'API ;
- `init: true` : tini en PID 1 relaie SIGTERM (drain de 5 s) et permet un vrai `kill -9` du processus, suivi d'un redémarrage par `restart: unless-stopped` (un `docker kill`, lui, est vu comme un arrêt manuel et ne redémarre pas, ce qui expliquait l'observation du TP 10) ;
- `stop_grace_period: 15s` > drain de 5 s ; limites CPU/mémoire ; 2 réplicas (`API_REPLICAS`) ;
- NGINX : `resolve` sur le nom `api`, keepalive, timeouts courts, bascule `proxy_next_upstream error timeout` (jamais pour un POST).

### 6.2 Kubernetes (bonus)

Le dossier `k8s/` contient : un namespace, une ConfigMap, le **Deployment** de l'API (3 réplicas ; sondes startup / readiness / liveness ; requests et limits ; `RollingUpdate maxUnavailable: 0, maxSurge: 1` ; `topologySpreadConstraints` par nœud et par zone et anti-affinité ; `runAsNonRoot`, `readOnlyRootFilesystem`, capabilities supprimées ; `terminationGracePeriodSeconds: 20`), le **Service**, le **PodDisruptionBudget** `minAvailable: 2` et les trois dépendances.

Le déploiement a été vérifié sur un cluster **minikube** local (profil dédié `shopflow`, 1 nœud) : application des manifestes, suppression d'un pod et mise à jour progressive pendant une boucle de requêtes, puis drain du nœud pour éprouver le PodDisruptionBudget.

```
$ minikube start -p shopflow ; minikube image load shopflow-api:latest shopflow-deps:latest
$ kubectl apply -f k8s/          # 11 ressources créées (validées aussi : kubernetes-validate 1.30 --strict)
$ kubectl -n shopflow rollout status deploy/shopflow-api
deployment "shopflow-api" successfully rolled out      # 3/3 pods prêts (startup + readiness OK)

# Test 1 — suppression d'un pod pendant une boucle de requêtes (45 s, pod client dans le cluster)
23:44:19 kubectl delete pod shopflow-api-...-bvq4d
23:44:22 ...-8ngjs 0/1 Running ; ...-bvq4d 0/1 Terminating (readiness en échec, drain) ; 2 autres 1/1
23:44:25 ...-8ngjs 1/1 Running  -> remplaçant prêt en ~6 s
RESULTAT {200: 258}                                    # 0 requête en échec

# Test 2 — rolling update (kubectl set env deploy/shopflow-api VERSION=2) pendant la boucle
23:45:16 -> 23:45:33  deployment "shopflow-api" successfully rolled out   (maxUnavailable: 0)
RESULTAT {200: 342}                                    # 0 requête en échec

# Test 3 — maintenance de nœud : kubectl drain shopflow (PDB minAvailable: 2, ALLOWED DISRUPTIONS = 1)
evicting pod shopflow-api-...-j596z                    # 1er pod évincé
error when evicting pods/"shopflow-api-...-qgrvg": Cannot evict pod as it would violate the pod's disruption budget.
error when evicting pods/"shopflow-api-...-rq7fx": Cannot evict pod as it would violate the pod's disruption budget.
=> 2 pods restent en service ; kubectl uncordon shopflow -> 3/3 Running
```

**Bilan** : aucune requête perdue ni pendant la recréation d'un pod (258/258), ni pendant le rolling update (342/342) ; le drain est bloqué par le PDB dès que 2 pods seulement resteraient disponibles. Sur un cluster à un seul nœud, l'anti-affinité et le topology spread ne peuvent pas répartir les pods (préférences `ScheduleAnyway`) : c'est précisément la limite de la question 3 du TP 10.

## 7. Chaos engineering : protocole et résultats

### 7.1 Protocole automatisé (`chaos/chaos.py`)

Chaque expérience suit la démarche du jour 1 : **mesure de référence** (15 s) → **injection** (via `/admin`, un pic de charge ou un `kill -9`) → **mesure pendant la panne** (20 s) avec **critère d'arrêt automatique** (SLI < 80 % sur 10 s glissantes : la panne est annulée) → **retour à la normale** → **mesure de la récupération** (20 s). La charge vient de 12 clients réguliers (≈ 60 req/s, 85 % de pages produit sur 100 SKU, 15 % de checkouts avec réessai utilisateur et même clé). Chaque expérience est jouée deux fois : l'API est redéployée avec `PATTERNS=off` (« avant »), puis `PATTERNS=on` (« après »).

- **Détection** : délai entre l'injection et la première réponse portant `X-Circuit-…: ouvert` (ouverture du breaker).
- **Récupération** : délai entre le retour à la normale et 10 réponses nominales consécutives (2xx, < 1 s, non dégradées).

### 7.2 Synthèse avant / après

| Exp. | Patterns | SLI réf. % | SLI panne % | dégradé % | p95 panne (s) | détection (s) | récup. (s) | arrêt |
|---|---|---|---|---|---|---|---|---|
| **E1** | sans | 100,0 | **77,1** | 0 | 0,17 | — | 0,2 | oui |
| | avec | 100,0 | **100,0** | 2 | 0,28 | — | 0,2 | non |
| **E2** | sans | 100,0 | **18,6** | 0 | 0,12 | — | 0,2 | oui |
| | avec | 100,0 | **100,0** | 85 | 0,11 | 0,5 | 5,1 | non |
| **E3** | sans | 100,0 | **17,5** | 0 | 2,91 | — | 1,5 | oui |
| | avec | 100,0 | **100,0** | 85 | 0,11 | 1,8 | 2,6 | non |
| **E4** | sans | 100,0 | **25,6** | 0 | 4,14 | — | 0,9 | oui |
| | avec | 100,0 | **100,0** | 85 | 0,31 | 2,3 | 3,1 | non |
| **E5** | sans | 100,0 | **96,4** | 0 | 0,17 | — | 0,2 | non |
| | avec | 100,0 | **100,0** | 0 | 0,17 | — | 0,2 | non |
| **E6** | sans | 100,0 | **99,0** | 0 | 0,77 | — | 0,5 | non |
| | avec | 100,0 | **68,4** | 0 | 0,80 | — | 0,6 | oui |
| **E7** | sans | 100,0 | **94,4** | 0 | 0,17 | — | 0,2 | non |
| | avec | 100,0 | **99,8** | 0 | 0,17 | — | 0,2 | non |

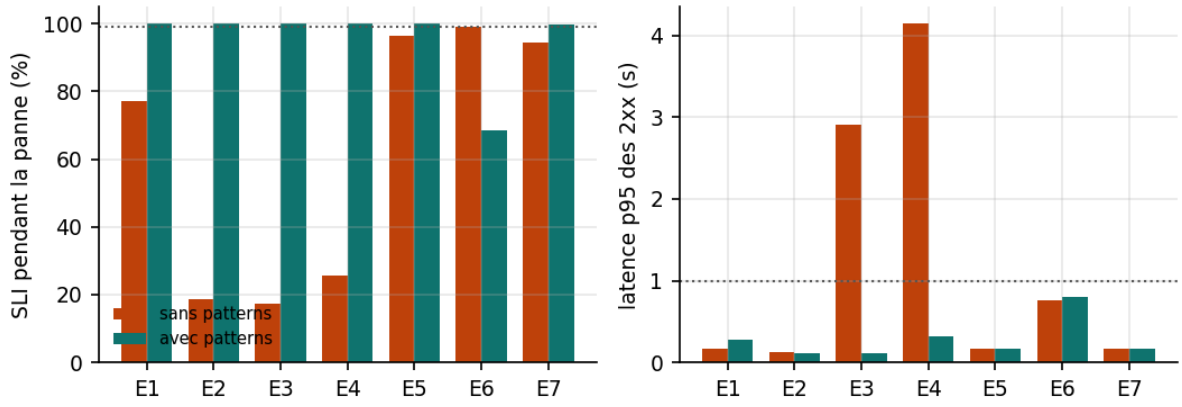



Figure 3 — SLI et latence p95 pendant la panne, sans (orange) et avec (vert) patterns. Pointillés : SLO 99 % et 1 s

### 7.3 Lecture des résultats

- **Sans patterns, toutes les pannes de dépendance franchissent le critère d'arrêt.** E1 à E4 tombent entre 17 % et 77 % de SLI et la panne est annulée automatiquement au bout de 10 s. Avec les patterns, le SLI reste à **100,0 %, 100,0 %, 100,0 % et 100,0 %**, sans arrêt de sécurité, et la p95 ne dépasse pas 0,31 s.
- **Latence (E3, E4)** : sans timeout, une dépendance lente se propage telle quelle (p95 2,91 s pour E3, 4,14 s pour E4) et occupe les 8 workers. Le timeout, le breaker et le bulkhead coupent ce couplage : la latence de la dépendance n'est plus celle de l'utilisateur.
- **Détection** : le breaker s'ouvre 0,50 s après l'arrêt du stock (E2) et 1,83 s après l'injection de latence (E3). La **récupération** complète (10 réponses nominales consécutives) prend 5,1 s en E2. Ce délai est surtout dû à la durée d'ouverture du breaker (5 s) : c'est le compromis entre ménager une dépendance qui redémarre et revenir vite au nominal.
- **Prix de la résilience : la dégradation.** Pendant E2 à E4, 85 à 85 % des réponses sont dégradées (stock issu du cache, page sans recommandations). Le SLI « servi » ne suffit donc pas à juger la qualité : la part dégradée est suivie à part, sur le tableau de bord et dans le tableau ci-dessus.
- **Paiement (E5)** : sans idempotence, 431 débits pour 394 paniers confirmés, soit **37 débits sans commande confirmée** (doubles débits après un réessai). Avec la clé d'idempotence et le retry : 458 débits pour 458 paniers confirmés, **0 écart**, 100,0 % de paniers payés.
- **Perte d'une instance (E7)** : avec une seule réplica, 60 requêtes échouent pendant le redémarrage. Avec deux réplicas et la bascule du gateway, il n'y en a plus que **2** (requêtes en cours sur l'instance tuée, dont des POST qu'on

ne rejoue jamais automatiquement) : SLI 99,82 %.

- **Pic de trafic (E6) : hypothèse invalidée**, détaillée en section 9.

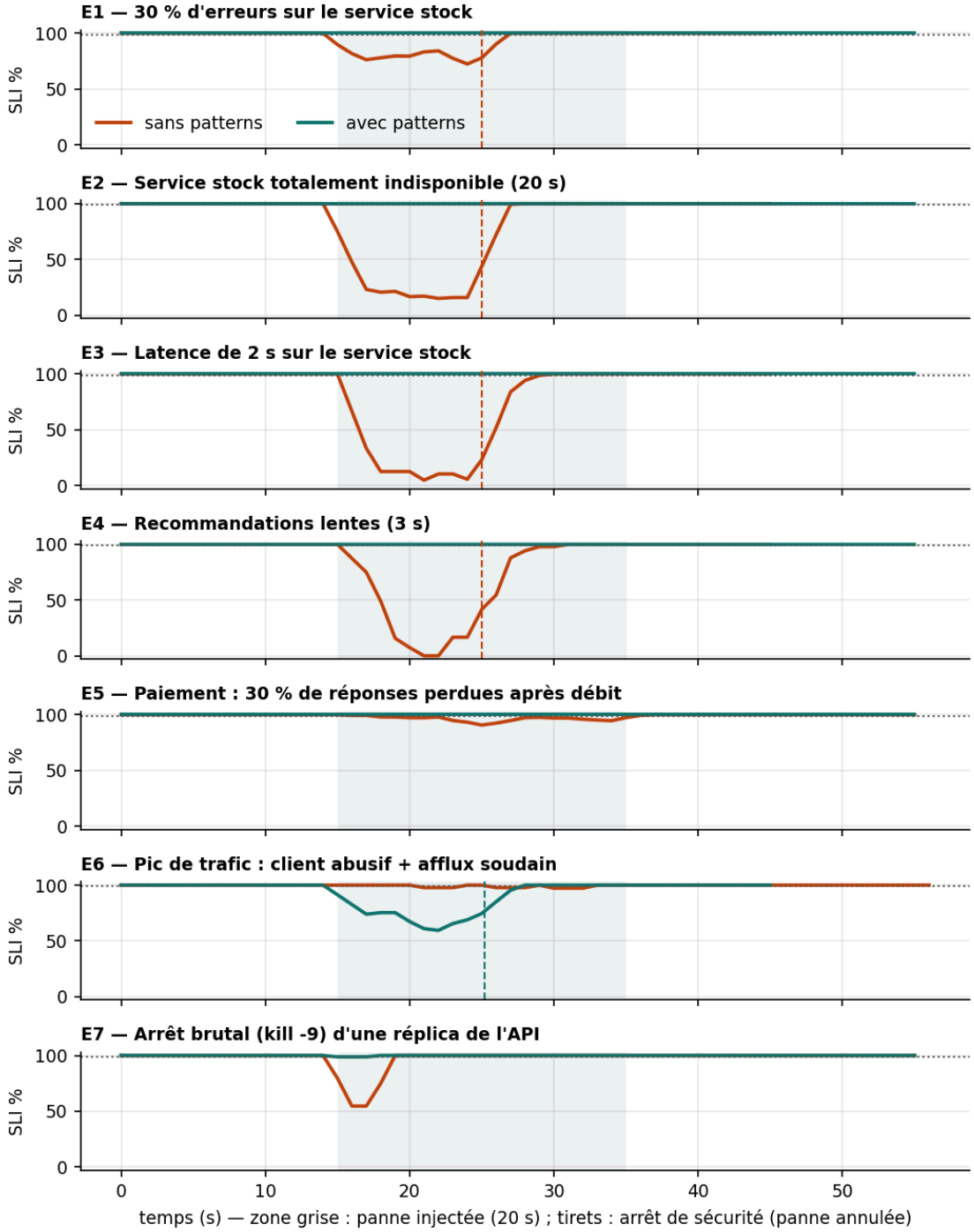



Figure 4 — SLI seconde par seconde (moyenne glissante sur 3 s). Zone grise : panne injectée (20 s) ; tirets : arrêt de sécurité (panne annulée)

## 8. Fiches d'expérience

### Expérience E1 — 30 % d'erreurs sur le service stock

| | |
|---|---|
| État stable (SLI mesuré) | SLI de référence 100,0 % (2xx en < 1 s), p95 0,17 s |
| Hypothèse | Le retry absorbe les erreurs transitoires : SLI > 99 %, aucune erreur visible. |
| Panne injectée | 30 % d'erreurs sur le service stock — durée 20 s, 12 clients réguliers (≈ 60 req/s) |
| Rayon d'impact | stock seulement ; toutes les pages produit (85 % du trafic) |
| Critère d'arrêt | SLI < 80 % sur 10 s glissantes → panne annulée automatiquement |
| Résultat sans patterns | SLI 77,1 %, p95 0,17 s, 5xx/réseau 128 — **arrêt de sécurité déclenché** |
| Résultat avec patterns | SLI **100,0 %**, p95 0,28 s, 2 % de réponses dégradées, détection — s, récupération 0,2 s |
| Mesures complémentaires | Première version : 62,6 % de réponses dégradées ; après réglage : 2,4 %. Erreurs 5xx visibles : 0. |
| Hypothèse validée ? | **Oui** pour le SLI ; la première version ouvrait le breaker sur du bruit (correctif 2, section 9). |
| Actions correctives | Breaker basé sur un taux d'échec (50 appels, 60 %, volume min. 20) au lieu de 20 appels / 50 %. |

### Expérience E2 — Service stock totalement indisponible (20 s)

| | |
|---|---|
| État stable (SLI mesuré) | SLI de référence 100,0 % (2xx en < 1 s), p95 0,17 s |
| Hypothèse | Le circuit s'ouvre en < 2 s ; le cache sert la dernière valeur : SLI > 95 %. |
| Panne injectée | Service stock totalement indisponible (20 s) — durée 20 s, 12 clients réguliers (≈ 60 req/s) |
| Rayon d'impact | stock ; toutes les pages produit |
| Critère d'arrêt | SLI < 80 % sur 10 s glissantes → panne annulée automatiquement |
| Résultat sans patterns | SLI 18,6 %, p95 0,12 s, 5xx/réseau 685 — **arrêt de sécurité déclenché** |
| Résultat avec patterns | SLI **100,0 %**, p95 0,11 s, 85 % de réponses dégradées, détection 0,5 s, récupération 5,1 s |
| Hypothèse validée ? | **Oui** : détection en moins de 2 s, SLI > 95 %. |
| Actions correctives | Préchauffer le cache des SKU les plus vendus au démarrage (un SKU jamais vu reste « inconnu »). |

### Expérience E3 — Latence de 2 s sur le service stock

| | |
|---|---|
| État stable (SLI mesuré) | SLI de référence 100,0 % (2xx en < 1 s), p95 0,17 s |
| Hypothèse | Timeout + breaker coupent la latence : p95 < 1 s et SLI > 95 %. |
| Panne injectée | Latence de 2 s sur le service stock — durée 20 s, 12 clients réguliers (≈ 60 req/s) |
| Rayon d'impact | stock ; pages produit ; risque de saturation des workers pour tout le trafic |
| Critère d'arrêt | SLI < 80 % sur 10 s glissantes → panne annulée automatiquement |
| Résultat sans patterns | SLI 17,5 %, p95 2,91 s, 5xx/réseau 0 — **arrêt de sécurité déclenché** |
| Résultat avec patterns | SLI **100,0 %**, p95 0,11 s, 85 % de réponses dégradées, détection 1,8 s, récupération 2,6 s |
| Hypothèse validée ? | **Oui**, après le correctif 3 (deadline) ; la version du TP 9 dépassait 1 s sur les premières requêtes. |
| Actions correctives | Budget global de 0,8 s partagé par les tentatives ; timeout du stock ramené à 0,3 s. |

### Expérience E4 — Recommandations lentes (3 s)

| | |
|---|---|
| État stable (SLI mesuré) | SLI de référence 100,0 % (2xx en < 1 s), p95 0,17 s |
| Hypothèse | La page produit est servie sans recommandations : SLI > 99 %. |
| Panne injectée | Recommandations lentes (3 s) — durée 20 s, 12 clients réguliers (≈ 60 req/s) |
| Rayon d'impact | reco ; bloc optionnel des pages produit |
| Critère d'arrêt | SLI < 80 % sur 10 s glissantes → panne annulée automatiquement |
| Résultat sans patterns | SLI 25,6 %, p95 4,14 s, 5xx/réseau 0 — **arrêt de sécurité déclenché** |
| Résultat avec patterns | SLI **100,0 %**, p95 0,31 s, 85 % de réponses dégradées, détection 2,3 s, récupération 3,1 s |
| Hypothèse validée ? | **Oui** : page servie sans recommandations, p95 inchangée. |
| Actions correctives | Aucune ; afficher « recommandations indisponibles » côté interface plutôt qu'une liste vide. |

### Expérience E5 — Paiement : 30 % de réponses perdues après débit

| | |
|---|---|
| **État stable (SLI mesuré)** | SLI de référence 100,0 % (2xx en < 1 s), p95 0,17 s |
| **Hypothèse** | Retry + clé d'idempotence : aucun double débit, > 99 % des paniers payés. |
| **Panne injectée** | Paiement : 30 % de réponses perdues après débit — durée 20 s, 12 clients réguliers (≈ 60 req/s) |
| **Rayon d'impact** | payment ; tous les checkouts (15 % du trafic) |
| **Critère d'arrêt** | SLI < 80 % sur 10 s glissantes → panne annulée automatiquement |
| **Résultat sans patterns** | SLI 96,4 %, p95 0,17 s, 5xx/réseau 37 |
| **Résultat avec patterns** | SLI **100,0 %**, p95 0,17 s, 0 % de réponses dégradées, détection — s, récupération 0,2 s |
| **Mesures complémentaires** | Sans patterns : 396 paniers, 431 débits, 37 en trop. Avec : 458 paniers, 458 débits, 0 en trop. |
| **Hypothèse validée ?** | **Oui** : aucun double débit, tous les paniers payés. |
| **Actions correctives** | Conserver les clés côté prestataire au moins 24 h ; réconciliation périodique des débits. |

### Expérience E6 — Pic de trafic : client abusif + afflux soudain

| | |
|---|---|
| **État stable (SLI mesuré)** | SLI de référence 100,0 % (2xx en < 1 s), p95 0,17 s |
| **Hypothèse** | Rate limiting + load shedding : les clients habituels gardent un SLI > 95 % et p95 < 1 s. |
| **Panne injectée** | Pic de trafic : client abusif + afflux soudain — durée 20 s, 12 clients réguliers (≈ 60 req/s) |
| **Rayon d'impact** | toutes les routes ; capacité des deux réplicas |
| **Critère d'arrêt** | SLI < 80 % sur 10 s glissantes → panne annulée automatiquement |
| **Résultat sans patterns** | SLI 99,0 %, p95 0,77 s, 5xx/réseau 0 |
| **Résultat avec patterns** | SLI **68,4 %**, p95 0,80 s, 0 % de réponses dégradées, détection — s, récupération 0,6 s — arrêt de sécurité déclenché |
| **Mesures complémentaires** | Client abusif : 1455 réponses 429 sur 1705 requêtes ; checkout réussi à 100 % pendant le pic. |
| **Hypothèse validée ?** | **Non** : le SLI des clients habituels baisse quand le délestage est actif (voir section 9). |
| **Actions correctives** | Délestage adaptatif fondé sur le temps d'attente (type CoDel) ; tests de charge en boucle ouverte ; autoscaling. |

### Expérience E7 — Arrêt brutal (kill -9) d'une réplica de l'API

| | |
|---|---|
| **État stable (SLI mesuré)** | SLI de référence 100,0 % (2xx en < 1 s), p95 0,17 s |
| **Hypothèse** | Le gateway bascule sur l'autre réplica ; < 1 % d'échecs ; redémarrage auto < 15 s. |
| **Panne injectée** | Arrêt brutal (kill -9) d'une réplica de l'API — durée 20 s, 12 clients réguliers (≈ 60 req/s) |
| **Rayon d'impact** | une réplica API sur deux ; requêtes en cours sur cette réplica |
| **Critère d'arrêt** | SLI < 80 % sur 10 s glissantes → panne annulée automatiquement |
| **Résultat sans patterns** | SLI 94,4 %, p95 0,17 s, 5xx/réseau 60 |
| **Résultat avec patterns** | SLI **99,8 %**, p95 0,17 s, 0 % de réponses dégradées, détection — s, récupération 0,2 s |
| **Mesures complémentaires** | Requêtes en échec : 60 (1 réplica) → 2 (2 réplicas). |
| **Hypothèse validée ?** | **Oui** : moins de 1 % d'échecs, redémarrage automatique. |
| **Actions correctives** | Rejouer automatiquement les GET interrompus au gateway (déjà fait) ; pour les POST, clé d'idempotence côté client. |

## 9. Faiblesses découvertes et corrections (avant / après)

Le chaos engineering sert d'abord à **apprendre** : quatre expériences ont révélé un comportement inattendu. Trois ont été corrigés et la correction a été re-mesurée ; le quatrième constat a invalidé une hypothèse et débouche sur une recommandation.

### Correctif 1 — le gateway amplifiait les erreurs (E1 sans patterns)

Premier passage de E1 : 30 % d'erreurs du stock ont produit **7,1 % de SLI** au lieu des ≈ 70 % attendus. Les journaux NGINX affichaient `no live upstreams`. Cause : `proxy_next_upstream … http_502` compte chaque 502 *applicatif* («

stock indisponible ») comme une panne de l'instance. Avec `max_fails=2`, les deux réplicas, pourtant saines, étaient éjectées tour à tour pendant 5 s. Correction : ne basculer que sur les erreurs de **connexion** (`error timeout`). Résultat : **7,1 %** → **77,1 %** à panne identique. Ce mode de défaillance a été ajouté à l'AMDEC (R11) : un mécanisme de résilience mal réglé peut lui-même provoquer la panne.

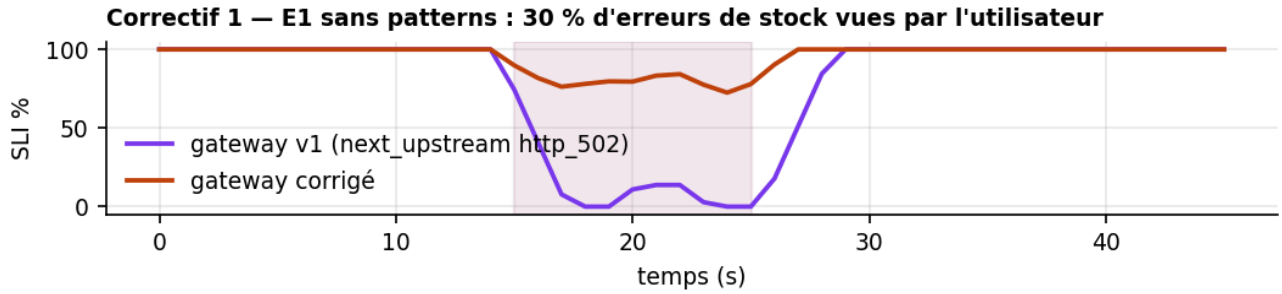



Figure 5 — E1 sans patterns : gateway initial (violet) vs corrigé (orange)

## Correctif 2 — le circuit breaker s'ouvrait sur du bruit (E1 avec patterns)

Avec 30 % d'erreurs indépendantes, le SLI restait à 100,0 %, mais **62,6 % des réponses étaient dégradées** : le breaker (fenêtre de 20 appels, seuil 50 %) s'ouvrait régulièrement. Chaque tentative étant comptée, la probabilité d'atteindre 10 échecs sur 20 avec p = 0,3 est d'environ 5 % *par fenêtre*, ce qui arrive en permanence à 60 req/s. Or le retry seul ramène l'échec à $0{,}3^3 \approx 2{,}7$ %. Correction : fenêtre de 50 appels, seuil 60 %, volume minimal 20 (probabilité de fausse ouverture ≈ 0,001 % sur une fenêtre pleine, 0,5 % sur une fenêtre encore partielle de 20 appels). Une panne franche ouvre toujours le breaker en moins d'une seconde (E2 : 0,50 s). Résultat : réponses dégradées **62,6 %** → **2,4 %**, SLI 100,0 %. **Contrepartie mesurée** : un breaker moins sensible s'ouvre un peu plus tard sur une panne de latence (E3 : 0,84 s → 1,83 s). Le SLI n'en souffre pas, car la deadline borne déjà chaque requête à 0,8 s.

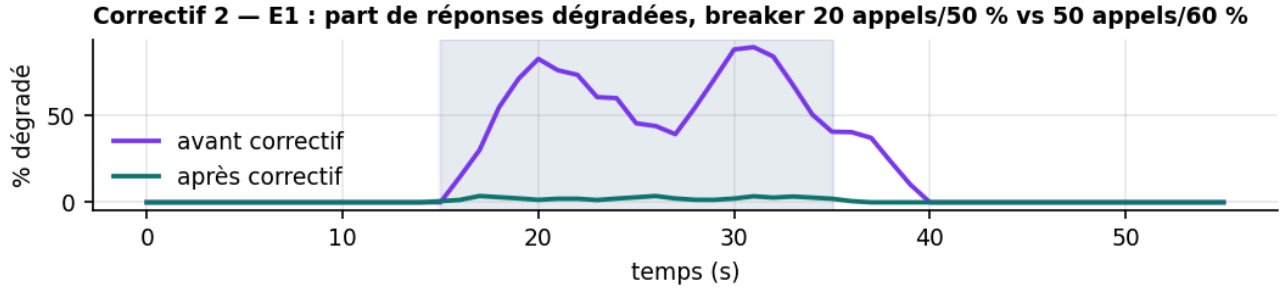



Figure 6 — E1 avec patterns : part de réponses dégradées avant / après le réglage du breaker

## Correctif 3 — retries sans budget global (E3)

Le client du TP 9 (timeout 0,5 s × 3 tentatives, sans budget global) peut mettre jusqu'à ≈ 1,6 s avant d'abandonner, plus que le SLO de 1 s, tant que le breaker n'est pas ouvert. Mesuré avec ces réglages (v1) : SLI 97,82 %, détection 3,14 s, récupération 4,8 s. Avec un **budget global de 0,8 s** partagé par toutes les tentatives et un timeout de 0,3 s : SLI **100,00 %**, détection 1,83 s, récupération 2,6 s. Le budget garantit par construction qu'aucune requête n'attend plus de 0,8 s sa dépendance, quel que soit l'état du breaker.

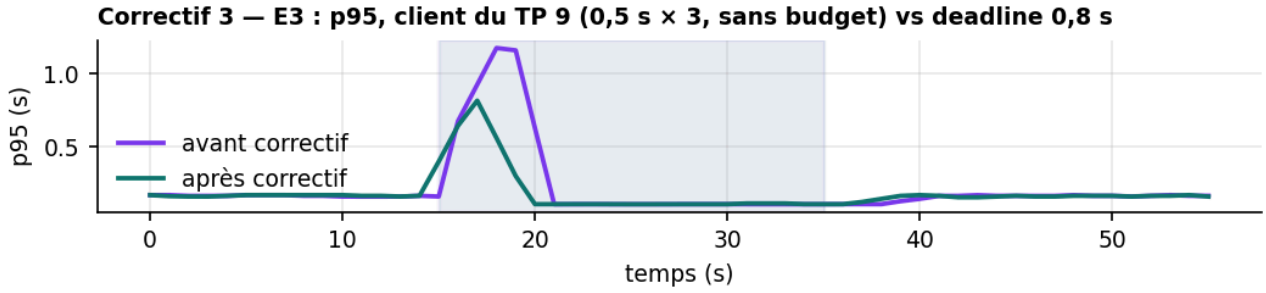



Figure 7 — E3 : latence p95 seconde par seconde, client du TP 9 vs budget global

### Constat 4 — E6 invalide l'hypothèse sur le délestage

Hypothèse : « rate limiting + load shedding gardent un SLI > 95 % aux clients habituels pendant un pic ». Mesures pendant le pic (client abusif à 25 threads et afflux de 60 nouveaux clients) :

| Variante | SLI clients habituels | débit utile (rép. bonnes/s) | p50 / p95 (s) | abusif : 429 | afflux : 503 |
|---|---|---|---|---|---|
| Sans patterns | 99,0 % | 14,9 | 0,70 / 0,77 | 0 | 0 |
| Shedder 12 places (v1) | 51,8 % | 15,8 | 0,36 / 0,48 | 855 | 1023 |
| Shedder 40 places (loi de Little) | 68,4 % | 15,6 | 0,56 / 0,80 | 1455 | 376 |

**Ce qui fonctionne** : le rate limiting écarte le client abusif à faible coût, et la réserve du shedder protège le checkout (succès 100 % pendant le pic). Recalibrer le shedder avec la loi de Little a fait passer le SLI de 51,8 % à 68,4 %.
**Ce que l'on a appris** : sans protection, le SLI semble meilleur, mais c'est un artefact de mesure. Nos clients de test attendent leur réponse avant d'envoyer la suivante (charge « en boucle fermée ») : quand le système ralentit, ils ralentissent aussi. Le débit des clients habituels passe de 52 à 15 req/s et la médiane de 0,13 s à 0,70 s, sans qu'aucune requête ne dépasse 1 s. De vrais utilisateurs, eux, continueraient d'arriver (boucle ouverte) : la file grandirait sans limite et le SLI s'effondrerait. Par ailleurs, l'afflux est composé de **clients légitimes** que le shedder ne peut pas distinguer des habitués : rejeter vite une partie du trafic est le comportement voulu, mais le SLI par requête le compte comme un échec. Le débit utile reste du même ordre (≈ 16 réponses bonnes/s) : le délestage ne crée pas de capacité, il choisit qui est servi.
**Actions** : (1) délestage adaptatif fondé sur le **temps d'attente** réel (rejeter quand l'attente dépasse ≈ 0,5 s, à la manière de CoDel) plutôt qu'un nombre fixe de places ; (2) tests de charge en **boucle ouverte** (débit imposé) pour mesurer honnêtement la surcharge ; (3) autoscaling horizontal sur la métrique `shopflow_inflight_requests` en Kubernetes.

## 10. Limites et perspectives

- **Dépendances simulées** : `flaky_service` produit des pannes indépendantes et sans état. Les vraies pannes sont corrélées (dégradation progressive, saturation liée à la charge). Toxiproxy permettrait d'injecter des pannes TCP réalistes (coupures partielles, paquets perdus, bande passante).
- **Tout tourne sur un seul hôte** : la redondance de l'API ne protège pas contre la perte de la machine. Le gateway est lui-même un SPOF en Compose. En production : Ingress répliqué, plusieurs nœuds et zones (manifestes fournis).
- **État local à chaque réplica** : token buckets, breakers et cache sont propres à chaque instance. Avec N réplicas, un client dispose de N fois son quota (TP 7, question 2), et chaque réplica doit détecter la panne de son côté. Un Redis partagé rendrait le rate limiting exact, au prix d'une nouvelle dépendance à protéger.
- **Charge en boucle fermée** générée depuis l'hôte Windows (≈ 60 à 300 req/s) : elle sous-estime la surcharge (constat E6).
- **Durées courtes** : 20 s de panne et quelques centaines de requêtes par phase. Les écarts de moins d'un point de SLI ne sont pas significatifs. Les alertes sont démontrées sur des fenêtres réduites (1 min / 5 min / 1 h) et non sur 30 jours.
- **Le checkout reste borné par le prestataire** (≈ 99 %) : aucun repli n'est possible pour un débit. Seule une file de paiements asynchrone (accepter la commande, payer plus tard) pourrait découpler le SLO du checkout de celui du prestataire.
- **Perspectives** : chaos continu en préproduction (expériences planifiées, critère d'arrêt branché sur l'alerte burn rate), puis en production sur une petite part du trafic ; Alertmanager pour router les alertes ; traces distribuées (OpenTelemetry) pour relier une requête lente à la dépendance en cause.